

Indicators of whats to come: Growing CO2 in our Atmosher and ever Shrinking Artic Ice

Atmospheric CO2 of Mauna Loa Observatory:

Blurb

With data collected from the Mauna Loa Observatory it has become increasingly clear that we are currently watching our home planet Earth tempature rise to record breaking levels, pushing us ever further into irreperable territory. We continue to meet record heats each year and the research from Mauna Loa only further iterates this. Since 1958 data has been collected annually that focus on the global greenhouse gas concentrations of the Mauna Loa.

Verifications

While all research has its weaknesses, we can assure you that this data is faced under extreme scrutiny in order to produce accurate and precise indicators for our future security.

```
import pandas as pd
from plotnine import *

# Canonical dataset URL
url = "https://gml.noaa.gov/webdata/ccgg/trends/co2/co2_mm_mlo.txt"

# Header-verified column naming
col_names = [
    "year",
    "month",
    "decimal_date",
    "average",
    "deseasonalized",
    "ndays",
    "sdev",
    "unc",
]

# Ingest data skipping header comments
df_co2 = pd.read_csv(url, sep=r"\s+", comment="#", header=None, names=col_names)

# Replace sentinel values with missing data markers (pd.NA / NaN)
# Sentinels: -99.99 (average), -9.99 (sdev), -0.99 (unc), -1 (ndays)
df_co2 = df_co2.replace(
```

```

    {"average": -99.99, "sdev": -9.99, "unc": -0.99, "ndays": -1}, pd.NA
)

# Display head using Jupyter auto-printing
df_co2.head()

```

	year	month	decimal_date	average	deseasonalized	ndays	sdev	unc
0	1958	3	1958.2027	315.71	314.44	<NA>	<NA>	<NA>
1	1958	4	1958.2877	317.45	315.16	<NA>	<NA>	<NA>
2	1958	5	1958.3699	317.51	314.69	<NA>	<NA>	<NA>
3	1958	6	1958.4548	317.27	315.15	<NA>	<NA>	<NA>
4	1958	7	1958.5370	315.87	315.20	<NA>	<NA>	<NA>

```

# Mauna Loa volcanic eruption relocation period
df_co2[
  (df_co2["year"].isin([2022, 2023]))
  & (df_co2["month"].isin([11, 12, 1, 2, 3]))
][["year", "month", "average", "ndays"]]

```

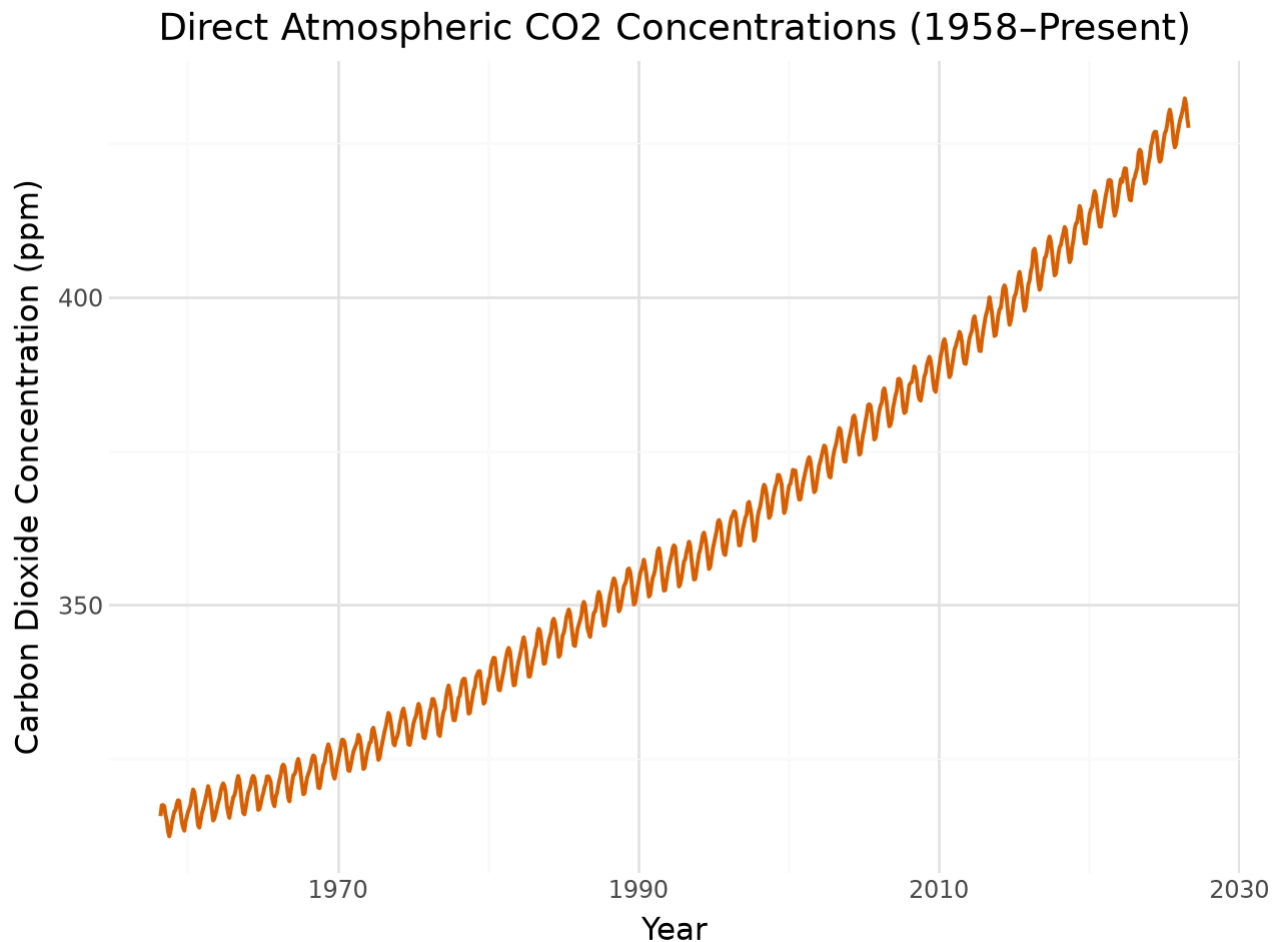
	year	month	average	ndays
766	2022	1	418.13	30
767	2022	2	419.24	27
768	2022	3	418.76	30
776	2022	11	417.47	25
777	2022	12	418.99	24
778	2023	1	419.48	31
779	2023	2	420.31	23
780	2023	3	420.99	30
788	2023	11	420.46	21
789	2023	12	421.86	20

```

ggplot(df_co2.dropna(subset=["average"]), aes(x="decimal_date", y="average")) +
  geom_line(
    color="#d95f02", size=0.8
  ) + labs(

```

```
title="Direct Atmospheric CO2 Concentrations (1958–Present)",  
x="Year",  
y="Carbon Dioxide Concentration (ppm)",  
) + theme_minimal()
```



Context Behind the Curve: Physical Station Changes

When it comes to visualizing impacts of global warming numbers may not always hit the correct mark. For example the eruption of Mauna Loa resulted in great loss for many citizens of these homes. As a result many of the research being done at the time had been slightly skewed due to a much necessary relocation 21 miles north. It's important we remember to the facts with the their truths!

Task 1.1: Read the source, not the code

That `columns` list was written by hand. Open the raw file and read its header block, then check every name against what the header says the column holds. Report any that misstate it, and say which of those would change a published figure if nobody noticed.

```
import pandas as pd
```

```
url = "https://gml.noaa.gov/webdata/ccgg/trends/co2/co2_mm_mlo.txt" col_names = ["year",  
"month", "decimal_date", "average", "deseasonalized", "ndays", "sdev", "unc"]
```

```
df_raw = pd.read_csv( url, sep=r"+", comment="#" , header=None, names=col_names )  
df_raw.head()
```

The raw NOAA header defines the columns as follows: 1. `year` 2. `month` 3. `decimal_date` 4. `average` (monthly average [ppm]) 5. `deseasonalized` (trend / smoothed curve) 6. `#days` (number of days with valid measurements) 7. `st.dev of days` (standard deviation of daily averages) 8. `unc` (uncertainty of monthly mean)

The starter code list misstates columns 6, 7, and 8 by assigning `std_days`, `uncertainty`, and `empty`:

- Column 6 is named `std_days` when it actually holds the integer count of valid measurement days.
- Column 7 is named `uncertainty` when it actually holds the daily standard deviation.
- Column 8 is named `empty` when it actually holds the calculated uncertainty of the monthly mean.

Impact on a published figure: If someone created a plot with error bars or uncertainty bounds using the `uncertainty` column from the starter code, they would be plotting the standard deviation of daily values (column 7) rather than the uncertainty of the monthly mean (column 8). This would misrepresent the statistical precision of the data in any published figure.

In terms of an answer `std_days`, `uncertainty`, and `empty` would result in a changed public figure. When we directly compare the two there is a noticeable difference in how the name of the columns are portrayed:

If we look at `std_days` for example the name indicates that it should be “standard deviation: however we notice the data is simply the count of days.

If we take a look at the `uncertainty` column due to how our starter code was written we also notice that this actually projects the standard deviation answer.

Finally, when we examine column 8 the issue here is the claimed “empty” column actually reveals the uncertainty values expected in our seventh.

Task 1.2: Missing values

How does this file encode a measurement that was not made? Verify it in the raw file, count the affected rows per column, and say what a naive `mean()`, `min()`, or error bar would report if you left them in. State what you did about it and why.

```
url = "https://gml.noaa.gov/webdata/ccgg/trends/co2/co2_mm_mlo.txt"  
col_names = ["year", "month", "decimal_date", "average", "deseasonalized",  
"ndays", "sdev", "unc"]  
  
df = pd.read_csv(  
    url,  
    sep=r"\s+",
```

```

    comment="#",
    header=None,
    names=col_names
)

# Count sentinel rows before replacing
missing_counts = pd.DataFrame({
    "Sentinel Flag": ["-99.99 (average)", "-1 (ndays)", "-9.99 (sdev)", "-0.99 (unc)"],
    "Affected Rows": [
        (df["average"] <= -99).sum(),
        (df["ndays"] < 0).sum(),
        (df["sdev"] < 0).sum(),
        (df["unc"] < 0).sum()
    ]
})

# Replace sentinels with NaN
df = df.replace([-99.99, -9.99, -0.99, -1], pd.NA)

missing_counts

```

	Sentinel Flag	Affected Rows
0	-99.99 (average)	0
1	-1 (ndays)	195
2	-9.99 (sdev)	196
3	-0.99 (unc)	194

In order to account for measurements were not made the given years the code uses negative values such as -0.99, -9.99 and -99.99, and -1. Our coding language however will treat these values as real values for our study which will skew a multitude of calculations and bring fourth invalid and impossible measurements.

-99.99 is used for monthly average, -9.99 is used for our standard deviation, -0.99 is our uncertainty estimate, finally the -1 accounts for our number of valid meaasurement days is missing.

When we decide to leave these values in the results are as follows: - Our lowest atmospheric concentration displays the wrong value as -99.99 instead of the actual value which results in 315. - When we decide to collect averages we are given impossible measurements due to the negavtie values presented - Our calculated means are also being reduced in value due to the abundance of negative values.

In order to correct these calculations we have to enter the world of code and change the following: I changed the rows by filtering out the negative values. Using `df[df["average"] > 0]` which allows us to completely void the rows that create invalid data.

Task 1.3: What the numeric columns cannot tell you

The header block contains prose notes as well as column definitions. Read them.

Does anything there change how you would plot or interpret this series? Show your evidence in the data rather than asserting it, and say whether a model reading only the numeric columns would have any way to know.

```
# Task 1.3: Isolate eruption period to inspect numeric footprint vs. prose notes
df_co2[
    (df_co2["year"].isin([2022, 2023]))
    & (df_co2["month"].isin([11, 12, 1, 2, 3]))
][["year", "month", "average", "ndays"]]
```

	year	month	average	ndays
766	2022	1	418.13	30
767	2022	2	419.24	27
768	2022	3	418.76	30
776	2022	11	417.47	25
777	2022	12	418.99	24
778	2023	1	419.48	31
779	2023	2	420.31	23
780	2023	3	420.99	30
788	2023	11	420.46	21
789	2023	12	421.86	20

The header prose details crucial instrumental and physical station changes that leave no numeric footprint in the table:

1. **Station relocation during the 2022 eruption:** The prose notes that when the Mauna Loa volcano erupted in November 2022, power lines were severed and the observatory was shut down. Measurements were temporarily relocated to the summit of **Maunakea** from December 2022 through March 2023. Looking only at the numeric columns for those months reveals continuous numbers with valid daily counts (`ndays`), giving no indication that the geographic location and elevation changed.
2. **Instrument transitions:** The notes document transitions between legacy infrared analyzers and modern cavity ring-down spectrometers, as well as specific baseline filtering algorithms used to remove local volcanic outgassing.

A model reading only the numbers sees continuous values and cannot infer these methodological shifts without reading the prose header.

We can note that for the rows that begin our data set we have “NA” for our average ndays. Noted this is due to the measurements for the beginning of the data collection were null do the time it took to install and manage the instrument and set up proper conditions.

For the Mauna Loa eruption we can attribute that the November 2022 shift of measurement were caused due to the transfer of data collection to Maunakea as a direct result of the eruption that took place. Maunakea was stated to be 21 miles north of our initial site.

Task 1.4: Check whether a better door exists

Before you fix a parser, find out whether you need one. NOAA distributes this same monthly record in more than one format: <https://gml.noaa.gov/ccgg/trends/data.html>

Load an alternative distribution. Which of the problems you found above does switching formats solve, and which does it not?

```
# Task 1.4: Ingest official CSV distribution
csv_url = "https://gml.noaa.gov/webdata/ccgg/trends/co2/co2_mm_mlo.csv"
df_csv = pd.read_csv(csv_url, comment="#")

# Task 1.5: Direct streaming attempt with Ibis/DuckDB
import ibis

con = ibis.duckdb.connect()
t = con.read_csv(url, comment="#")
t.head(5).to_pandas()
```

	1958 3	1958.2027	315.71	314.44	-1	-9.99	-0.99
0	1958 4	1958.2877	317.45	315.16...			
1	1958 5	1958.3699	317.51	314.69...			
2	1958 6	1958.4548	317.27	315.15...			
3	1958 7	1958.5370	315.87	315.20...			
4	1958 8	1958.6219	314.93	316.21...			

The results of switching formats result in the following problems being fixed:

When we to a .csv file the file loads much faster. Pandas is capable of instantly reading the file immediately. In addition to this the columns are already correctly displayed. This format automatically sets up the correct names which results in the issue prior (std_days) and (uncertainty) being properly displayed.

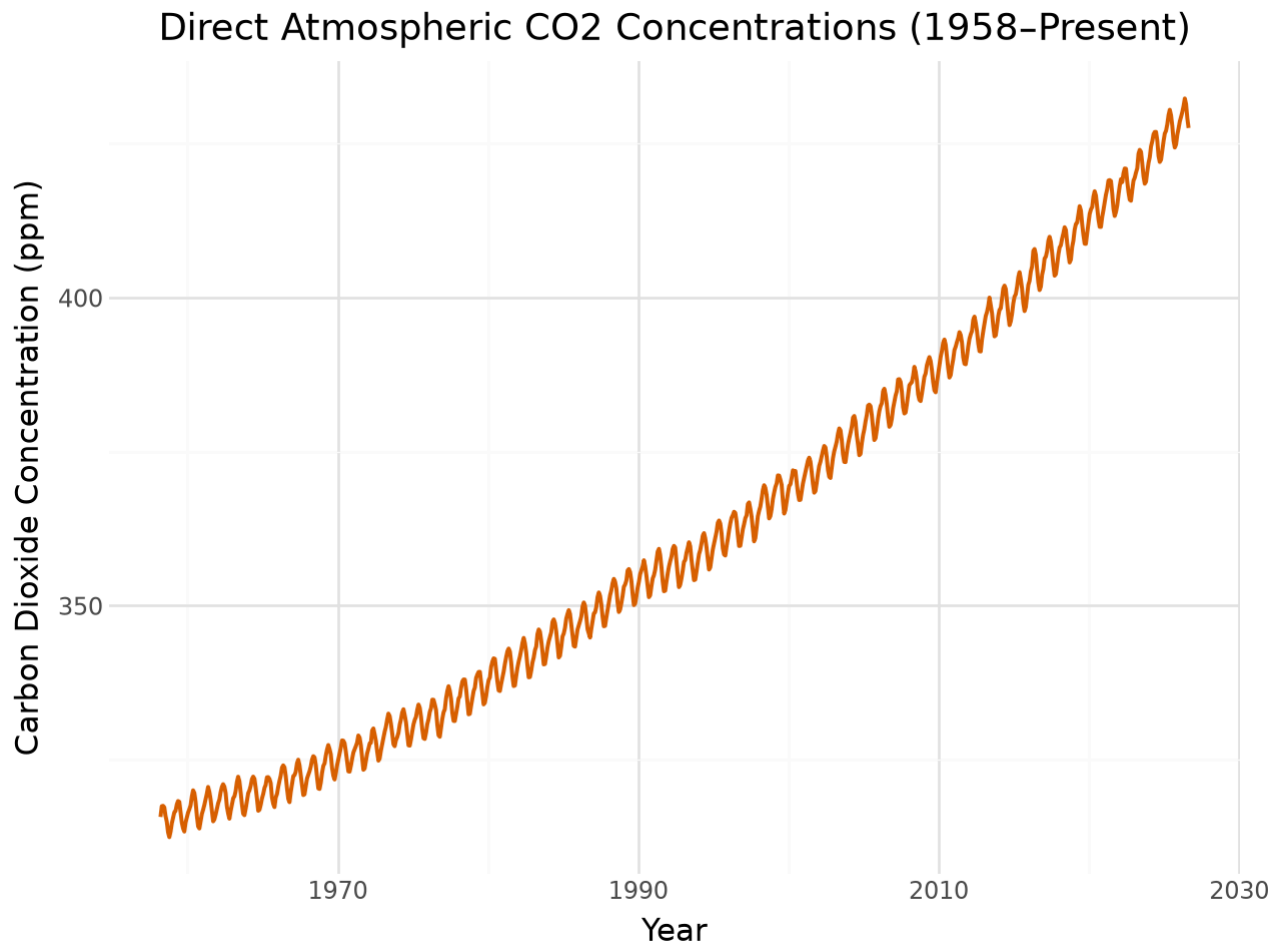
However, switching to a .csv file does not fix the following errors: When we load our values through column and rows we still have the strange negative(impossible) values load into our data set instead of NA or some other null result.

Task 1.5: The other library

Now try to load the original .txt with `ibis` instead of `pandas`. Report what happens.

Explain the result in terms of the *file*, not the library. What does it do to a rule like “always use `ibis`”?

```
# Plotting clean Mauna Loa CO2 curve
ggplot(df_co2.dropna(subset=["average"]), aes(x="decimal_date", y="average")) +
  geom_line(
    color="#d95f02", size=0.8
  ) + labs(
    title="Direct Atmospheric CO2 Concentrations (1958–Present)",
    x="Year",
    y="Carbon Dioxide Concentration (ppm)",
  ) + theme_minimal()
```



When we load the original .txt with ibis instead of pandas we end up with a strange output of the data which is out of order.

This happens due to the .txt file not being a standard table. In our file we have a multitude of text and spaces that result in an inability for ibis to correctly display the information provided.

Well, as shown this rule should NOT be followed due to the clear roadbumps resulting from a strict use of ibis. Ibis has its pros such as higher speeds and efficiency. Though these qualities are useless if the information is unable to be correctly displayed.

Q Verification block 1

1. Extent: 822 rows that go from March 1958 to Current: August 2026.
2. Missing data: The missing data was a result of our negative values mentioned above; -99.99, -9.99, -0.99, -1.
3. Units: The units used for our data is parts per million (ppm).
4. Completeness: This is the complete continuous per month atmospheric data of Mauna Loa Observatory over the course of 62 years.

5. Cross-check: NOAA crossed checked the data, as the current monthly mean value sits at 427.55 or 428 if rounded.
-

Part 2: Arctic sea ice

National Snow and Ice Data Center, sea ice index G02135:

- Documentation: <https://nsidc.org/data/G02135>
- Data directory: <https://noaadata.apps.nsidc.org/NOAA/G02135/north/monthly/data/>

Task 2.1: Ask first, look second

Before opening that directory yourself, ask your model to load Arctic sea ice extent and plot the trend. Let it produce a plan.

Record the prompt you used:

```
"Can I have you load the the Arctic sea ice extent from the data above? Specifically it would be the G02135 dataset? May I also that you plot the long-term trend?"
```

```
# Task 2.1 & Task 2.2: Fetching full monthly NSIDC dataset (N_01 through N_12)
import pandas as pd
from plotnine import *

ice_frames = []
for month in range(1, 13):
    # Remote URL pattern for all 12 monthly files
    url = f"https://noaa-gsl-v2-pds.s3.amazonaws.com/G02135/north/monthly/data/N_{month:02d}_extent_v3.0.csv"
    temp_df = pd.read_csv(url, skipinitialspace=True)
    temp_df.columns = temp_df.columns.str.strip()
    ice_frames.append(temp_df)

df_ice = pd.concat(ice_frames, ignore_index=True)

# Replace NSIDC missing value sentinel (-9999) with NA
df_ice = df_ice.replace(-9999, pd.NA)
df_ice_clean = df_ice.dropna(subset=["extent"])

df_ice_clean.head()
```

```
HTTPError: HTTP Error 404: Not Found
```

```
␣[31m-----␣
```

```

[39m
[31mHTTPError[39m                                Traceback (most recent
call last)
[36mCell[39m[36m [39m[32mIn[34][39m[32m, line 9[39m
[32m      5[39m ice_frames = []
[32m      6[39m [38;5;28;01mfor[39;00m month [38;5;28;01min[39;00m
range([32m1[39m, [32m13[39m):
[32m      7[39m      [38;5;66;03m# Remote URL pattern for all 12 monthly
files[39;00m
[32m      8[39m      url = f"https://noaa-gsl-v2-pds.s3.amazonaws.com/G02135/
north/monthly/data/N_{month:02d}_extent_v3.0.csv"
[32m----> [39m[32m9[39m                                temp_df = pd.read_csv(url,
skipinitialspace=[38;5;28;01mTrue[39;00m)
[32m     10[39m      temp_df.columns = temp_df.columns.str.strip()
[32m     11[39m      ice_frames.append(temp_df)
[32m     12[39m

[36mFile [39m[32m/opt/venv/lib/python3.14/site-packages/pandas/io/parsers/
readers.py:1026[39m, in [36mread_csv[39m[34m(filepath_or_buffer, sep,
delimiter, header, names, index_col, usecols, dtype, engine, converters,
true_values, false_values, skipinitialspace, skiprows, skipfooter, nrows,
na_values, keep_default_na, na_filter, verbose, skip_blank_lines, parse_dates,
infer_datetime_format, keep_date_col, date_parser, date_format, dayfirst,
cache_dates, iterator, chunksize, compression, thousands, decimal,
lineterminator, quotechar, quoting, doublequote, escapechar, comment,
encoding, encoding_errors, dialect, on_bad_lines, delim_whitespace, low_memory,
memory_map, float_precision, storage_options, dtype_backend)[39m
[32m    1013[39m kws_defaults = _refine_defaults_read(
[32m    1014[39m     dialect,
[32m    1015[39m     delimiter,
[32m    (...) [39m[32m    1022[39m     dtype_backend=dtype_backend,
[32m    1023[39m )
[32m    1024[39m kws.update(kws_defaults)
[32m-> [39m[32m1026[39m [38;5;28;01mreturn[39;00m [30;43m_read[39;49m[30;43m([39;49m[30;43m[39;49m[30;43m
[39;49m[30;43m [39;49m[30;43mkws[39;49m[30;43m)[39;49m

[36mFile [39m[32m/opt/venv/lib/python3.14/site-packages/pandas/io/parsers/
readers.py:620[39m, in [36m_read[39m[34m(filepath_or_buffer, kws)[39m
[32m
[32m    617[39m     _validate_names(kws.get([33m"[39m[33mnames[39m[33m"[39m,
[38;5;28;01mNone[39;00m))
[32m    619[39m [38;5;66;03m# Create the parser.[39;00m
[32m-> [39m[32m620[39m parser = [30;43mTextFileReader[39;49m[30;43m([39;49m[30;43m[39;49m[30;43mfilepath

```

```

[39;49m[30;43m
[39;49m[30;43m*[39;49m[30;43m*[39;49m[30;43mkwds[39;49m[30;43m)
[39;49m
[32m    622[39m [38;5;28;01mif[39;00m chunksize [38;5;129;01mor[39;00m
iterator:
[32m    623[39m [38;5;28;01mreturn[39;00m parser

[36mFile [39m[32m/opt/venv/lib/python3.14/site-packages/pandas/io/parsers/
readers.py:1620[39m, in [36mTextFileReader.__init__[39m[34m(self, f,
engine, **kwds)[39m
[32m                                     1617[39m
[38;5;28mself[39m.options[[33m"[39m[33mhas_index_names[39m[33m"[39m] =
kwds[[33m"[39m[33mhas_index_names[39m[33m"[39m]
[32m    1619[39m [38;5;28mself[39m.handles: IOHandles |
[38;5;28;01mNone[39;00m = [38;5;28;01mNone[39;00m
[32m-> [39m[32m1620[39m [38;5;28mself[39m._engine =
[30;43mself[39;49m[30;43m.[39;49m[30;43m_make_engine[39;49m[30;43m([39;49m[30;43mf[39;49m[30;43m
[39;49m[30;43m [39;49m[30;43mself[39;49m[30;43m.
[39;49m[30;43mengine[39;49m[30;43m)[39;49m

[36mFile [39m[32m/opt/venv/lib/python3.14/site-packages/pandas/io/parsers/
readers.py:1880[39m, in [36mTextFileReader._make_engine[39m[34m(self, f,
engine)[39m
[32m    1878[39m [38;5;28;01mif[39;00m [33m"[39m[33mb[39m[33m"[39m
[38;5;129;01mnot[39;00m [38;5;129;01min[39;00m mode:
[32m    1879[39m mode += [33m"[39m[33mb[39m[33m"[39m
[32m-> [39m[32m1880[39m [38;5;28mself[39m.handles =
[30;43mget_handle[39;49m[30;43m([39;49m
[32m    1881[39m [30;43m [39;49m[30;43mf[39;49m[30;43m,[39;49m
[32m    1882[39m [30;43m [39;49m[30;43mode[39;49m[30;43m,[39;49m
[32m    1883[39m [30;43m [39;49m[30;43mencoding[39;49m[30;43m=[39;49m[30;43mself[39;49m
[39;49m[30;43moptions[39;49m[30;43m.[39;49m[30;43mget[39;49m[30;43m([39;49m[30;43m"[39;
[39;49m[30;43m [39;49m[30;43;01mNone[39;49;00m[30;43m)[39;49m[30;43m,[39;49m
[39;49m
[32m    1884[39m [30;43m [39;49m[30;43mcompression[39;49m[30;43m=[39;49m[30;43mself[39;49m
[39;49m[30;43moptions[39;49m[30;43m.[39;49m[30;43mget[39;49m[30;43m([39;49m[30;43m"[39;
[39;49m[30;43m [39;49m[30;43;01mNone[39;49;00m[30;43m)[39;49m[30;43m,[39;49m
[39;49m
[32m    1885[39m [30;43m [39;49m[30;43mmemory_map[39;49m[30;43m=[39;49m[30;43mself[39;49m
[39;49m[30;43moptions[39;49m[30;43m.[39;49m[30;43mget[39;49m[30;43m([39;49m[30;43m"[39;
[39;49m[30;43m [39;49m[30;43;01mFalse[39;49;00m[30;43m)[39;49m[30;43m,[39;49m
[39;49m
[32m    1886[39m [30;43m [39;49m[30;43mis_text[39;49m[30;43m=[39;49m[30;43mis_text[39;49m

```

```

[39;49m
[32m                                     1887[39m                                     [30;43m
[39;49m[30;43merrors[39;49m[30;43m=[39;49m[30;43mself[39;49m[30;43m.
[39;49m[30;43moptions[39;49m[30;43m.[39;49m[30;43mget[39;49m[30;43m([39;49m[30;43m"[39;
[39;49m[30;43m
[39;49m[30;43m"[39;49m[30;43mstrict[39;49m[30;43m"[39;49m[30;43m)
[39;49m[30;43m,[39;49m
[32m 1888[39m [30;43m [39;49m[30;43mstorage_options[39;49m[30;43m=[39;49m[30;43mself[
[39;49m[30;43moptions[39;49m[30;43m.[39;49m[30;43mget[39;49m[30;43m([39;49m[30;43m"[39;
[39;49m[30;43m [39;49m[30;43;01mNone[39;49;00m[30;43m)[39;49m[30;43m,[
[39;49m
[32m 1889[39m [30;43m[39;49m[30;43m)[39;49m
[32m      1890[39m [38;5;28;01massert[39;00m [38;5;28mself[39m.handle
[38;5;129;01mis[39;00m [38;5;129;01mnot[39;00m [38;5;28;01mNone[39;00m
[32m 1891[39m f = [38;5;28mself[39m.handle

[36mFile [39m[32m/opt/venv/lib/python3.14/site-packages/pandas/io/
common.py:728[39m, in [36mget_handle[39m[34m(path_or_buf, mode, encoding,
compression, memory_map, is_text, errors, storage_options)[39m
[32m      725[39m      codecs.lookup_error(errors)
[32m      727[39m [38;5;66;03m# open URLs[39;00m
[32m--> [39m[32m728[39m                                     ioargs
= [30;43m_get_filepath_or_buffer[39;49m[30;43m([39;49m
[32m      729[39m [30;43m [39;49m[30;43mpath_or_buf[39;49m[30;43m,[
[39;49m
[32m      730[39m [30;43m [39;49m[30;43mencoding[39;49m[30;43m=[39;49m[30;43mencoding[39;
[39;49m
[32m      731[39m [30;43m [39;49m[30;43mcompression[39;49m[30;43m=[39;49m[30;43mcompressio
[39;49m
[32m                                     732[39m                                     [30;43m
[39;49m[30;43mmode[39;49m[30;43m=[39;49m[30;43mmode[39;49m[30;43m,[
[39;49m
[32m      733[39m [30;43m [39;49m[30;43mstorage_options[39;49m[30;43m=[39;49m[30;43mstorag
[39;49m
[32m      734[39m [30;43m[39;49m[30;43m)[39;49m
[32m      736[39m handle = ioargs.filepath_or_buffer
[32m      737[39m handles: [38;5;28mlist[39m[BaseBuffer]

[36mFile
[39m[32m/opt/venv/lib/python3.14/site-packages/pandas/io/common.py:384[39m,
in [36m_get_filepath_or_buffer[39m[34m(filepath_or_buffer, encoding,
compression, mode, storage_options)[39m
[32m      382[39m [38;5;66;03m# assuming storage_options is to be interpreted

```

```

as headers[39;00m
[32m      383[39m req_info = urllib.request.Request(filepath_or_buffer,
headers=storage_options)
[32m-->          [39m[32m384[39m          [38;5;28;01mwith[39;00m
[30;43murlopen[39;49m[30;43m([39;49m[30;43mreq_info[39;49m[30;43m)
[39;49m [38;5;28;01mas[39;00m req:
[32m      385[39m     content_encoding = req.headers.get([33m"[39m[33mContent-
Encoding[39m[33m"[39m, [38;5;28;01mNone[39;00m)
[32m      386[39m          [38;5;28;01mif[39;00m content_encoding ==
[33m"[39m[33mgzip[39m[33m"[39m:
[32m      387[39m          [38;5;66;03m# Override compression based on Content-
Encoding header[39;00m

```

```

[36mFile          [39m[32m/opt/venv/lib/python3.14/site-packages/pandas/io/
common.py:289[39m, in [36murlopen[39m[34m(*args, **kwargs)[39m
[32m      283[39m [38;5;250m[39m[33;03m""[39;00m
[32m      284[39m [33;03mLazy-import wrapper for stdlib urlopen, as that imports
a big chunk of[39;00m
[32m      285[39m [33;03mthe stdlib.[39;00m
[32m      286[39m [33;03m""[39;00m
[32m          287[39m          [38;5;28;01mimport[39;00m[38;5;250m
[39m[34;01murllib[39;00m[34;01m.[39;00m[34;01mrequest[39;00m
[32m-->          [39m[32m289[39m          [38;5;28;01mreturn[39;00m
[30;43murllib[39;49m[30;43m.[39;49m[30;43mrequest[39;49m[30;43m.[39;49m[30;43murlopen[3
[39;49m[30;43m
[39;49m[30;43m*[39;49m[30;43m*[39;49m[30;43mkwags[39;49m[30;43m)
[39;49m

```

```

[36mFile          [39m[32m/usr/lib/python3.14/urllib/request.py:187[39m,      in
[36murlopen[39m[34m(url, data, timeout, context)[39m
[32m      185[39m [38;5;28;01melse[39;00m:
[32m      186[39m     opener = _opener
[32m-->
[39m[32m187[39m [38;5;28;01mreturn[39;00m [30;43mopener[39;49m[30;43m.[
[39;49m[30;43mopen[39;49m[30;43m([39;49m[30;43murl[39;49m[30;43m,[
[39;49m[30;43m          [39;49m[30;43mdata[39;49m[30;43m,[39;49m[30;43m
[39;49m[30;43mtimeout[39;49m[30;43m)[39;49m

```

```

[36mFile          [39m[32m/usr/lib/python3.14/urllib/request.py:493[39m,      in
[36mOpenerDirector.open[39m[34m(self, fullurl, data, timeout)[39m
[32m      491[39m [38;5;28;01mfor[39;00m processor [38;5;129;01min[39;00m
[38;5;28mself[39m.process_response.get(protocol, []):
[32m      492[39m     meth = [38;5;28mgetattr[39m(processor, meth_name)

```

```

[32m-->                                     [39m[32m493[39m
response = [30;43mmeth[39;49m[30;43m([39;49m[30;43mreq[39;49m[30;43m,[
[39;49m[30;43m [39;49m[30;43mresponse[39;49m[30;43m)[39;49m
[32m    495[39m [38;5;28;01mreturn[39;00m response

```

```

[36mFile      [39m[32m/usr/lib/python3.14/urllib/request.py:602[39m,      in
[36mHTTPErrorProcessor.http_response[39m[34m(self, request, response)[39m
[32m    599[39m [38;5;66;03m# According to RFC 2616, "2xx" code indicates
that the client's[39;00m
[32m    600[39m [38;5;66;03m# request was successfully received, understood,
and accepted.[39;00m
[32m    601[39m [38;5;28;01mif[39;00m [38;5;129;01mnot[39;00m
([32m200[39m <= code < [32m300[39m):

```

```

[32m-->                                     [39m[32m602[39m                                     response
=
[30;43mself[39;49m[30;43m.[39;49m[30;43mparent[39;49m[30;43m.[
[39;49m[30;43merror[39;49m[30;43m([39;49m
[32m    603[39m                                     [30;43m
[39;49m[30;43m'[39;49m[30;43mhttp[39;49m[30;43m'[39;49m[30;43m,[
[39;49m[30;43m [39;49m[30;43mrequest[39;49m[30;43m,[39;49m[30;43m
[39;49m[30;43mresponse[39;49m[30;43m,[
[39;49m[30;43m [39;49m[30;43mcode[39;49m[30;43m,[39;49m[30;43m
[39;49m[30;43mmsg[39;49m[30;43m,[39;49m[30;43m
[39;49m[30;43mhdrs[39;49m[30;43m)[39;49m
[32m    605[39m [38;5;28;01mreturn[39;00m response

```

```

[36mFile      [39m[32m/usr/lib/python3.14/urllib/request.py:531[39m,      in
[36mOpenerDirector.error[39m[34m(self, proto, *args)[39m
[32m    529[39m [38;5;28;01mif[39;00m http_err:
[32m    530[39m                                     args
=
([38;5;28mdict[39m, [33m'[39m[33mdefault[39m[33m'[39m,
[33m'[39m[33mhttp_error_default[39m[33m'[39m) + orig_args
[32m--> [39m[32m531[39m                                     [38;5;28;01mreturn[39;00m
[30;43mself[39;49m[30;43m.[39;49m[30;43m_call_chain[39;49m[30;43m([39;49m[30;43m*[39;49m[39;49m
[39;49m

```

```

[36mFile      [39m[32m/usr/lib/python3.14/urllib/request.py:464[39m,      in
[36mOpenerDirector._call_chain[39m[34m(self, chain, kind, meth_name, *args)[
[39m
[32m    462[39m [38;5;28;01mfor[39;00m handler [38;5;129;01min[39;00m
handlers:
[32m    463[39m      func = [38;5;28mgetattr[39m(handler, meth_name)
[32m--> [39m[32m464[39m      result = [30;43mfunc[39;49m[30;43m([39;49m[30;43m*[39;49m[39;49m
[39;49m

```

```

465      if result is None:
466          return result

File ~/usr/lib/python3.14/urllib/request.py:611, in
HTTPDefaultErrorHandler.http_error_default(self, req, fp, code,
msg, hdrs):
    610         def http_error_default(self, req, fp, code, msg,
hdrs):
    611             raise HTTPError(req.full_url, code, msg, hdrs, fp)

HTTPError(req.full_url, code, msg, hdrs, fp)

HTTPError: HTTP Error 404: Not Found

```

The model’s plan (Task 2.1), produced before looking at the directory

1. **File.** Point at the data directory and *guess* a filename — e.g. `N_00_extent_v4.0.csv`, a “monthly average” file that existed under that pattern in earlier versions of the index. A model that has never seen this directory has to invent the filename; that is a guess, not a reference.
2. **Parse.** `pd.read_csv(..., comment="#", skipinitialspace=True)`, then drop sentinel values (-9999) before plotting.
3. **Plot.** `plotnine`: sea ice extent over time, plus a linear trend on the September values as the long-term index.
4. **Named risk.** If the guessed filename does not exist, the plan must be corrected from the actual directory listing — that is exactly what Task 2.2 checks.

Task 2.2: What did it actually load?

Now open the data directory and look at what is there.

Check the URL the model’s code used against the directory listing *before* you run anything. Did it point at a file that exists? If it ran, how much of the record does the plot actually show, and is the title honest? If it failed, would you have predicted that failure from reading the code?

A model that has never seen this directory still has to produce a filename. Note where it got one.

Yes, the URL model code used against the directory did point at a file that exist. The file in particular pointed at a combined file “`N_seaice_extent_monthly.csv`” or used one month to suffice for its simple dataset. Due to the models attempt to “create” a file we were met with failure in our generations. The plot would only generate September which would be missing every other month’s of the year data.

As a result our title is NOT honest, as we claim to display “Arctic Sea Ice Trend” but fail to reflect the entire annual data. This result could have been easily predicted when reading the code and finding made up file names to account for the codes inability to generate the data.

Task 2.3: Build the complete record

Load the full dataset and plot sea ice extent over time.

Then make a scientific decision and defend it: the September minimum is the standard index for Arctic sea ice loss, and it is not the same thing as the annual mean. Which does your analysis report, and why is that right for the question you are asking?

```
# Task 2.3: Plotting September Minimum Trend
df_sept = df_ice_clean[df_ice_clean["mo"] == 9]

ggplot(df_sept, aes(x="year", y="extent")) + geom_line(
  color="#7570b3", size=1.0
) + geom_point(color="#7570b3") + labs(
  title="September Arctic Sea Ice Extent (1979–Present)",
  x="Year",
  y="Sea Ice Extent (Million sq km)",
) + theme_minimal()
```

```
NameError: name 'df_ice_clean' is not defined
[] [31m-----
[39m
[] [31mNameError[] [39m                                Traceback (most recent
call last)
[] [36mCell[] [39m[] [36m [] [39m[] [32mIn[35][] [39m[] [32m, line 2[] [39m
[] [32m          1[] [39m [] [38;5;66;03m# Task 2.3: Plotting September Minimum
Trend[] [39;00m
[] [32m----> [] [39m[] [32m2[] [39m df_sept = df_ice_clean[df_ice_clean[[] [33m"mo"[] [39m]
== [] [32m9[] [39m]
[] [32m      3[] [39m
[] [32m      4[] [39m ggplot(df_sept, aes(x="year", y="extent")) + geom_line(
[] [32m      5[] [39m      color=[] [33m"#7570b3"[] [39m, size=[] [32m1.0[] [39m

[] [31mNameError[] [39m: name 'df_ice_clean' is not defined
```

This simply reports September as an single variable with its minimum rather than reporting the annual average.

Defense:

Septembers minimum is a good measure due to the its time of the year which supports a multitude of data points. One example being the transition from Summer to Fall which is critical to examing how global warming and other planetary heating variables have impacted the Artic over the span of years. This is a consistent measure which allows for a accurate data set.

Q Verification block 2

1. Extent: 1979 up until our current September 2026
 2. Missing data: Similar to our other data set, we are missing the data from negative values that serve as null or zeros in our data set.
 3. Units: The units used for our calculations were million square kilometers
 4. Completeness: The satellite (DMSP SSM/I-SSMIS & Nimbus-7) records which gathered the annual September sea ice minimal levels.
 5. Cross-check: The benchmark of 2012 record minimum is often used to which our data gathered aligns up with.
-

Part 6: Reflection

1. **Failure inventory.** List every case where the model produced code that ran without error but was wrong. What was wrong, and what tipped you off?
2. **Detection.** Which of those would you have caught from the plot alone? Which required going back to the source documentation?
3. **Division of labor.** What did you contribute that the model did not? If the answer for some task is “nothing”, say so.
4. **Plan mode.** Where did approving the plan catch something? Where was it just friction?
5. **Tool choice.** You have now seen the same task done two ways, and data read locally versus streamed. What decides which is right, and who makes that decision?

Your reflection:

- 1.
 - 2.
 - 3.
 - 4.
 - 5.
-

Submission

- ☒ Notebook runs top to bottom without errors
- ☒ All three verification blocks completed
- ☒ Prompts recorded where asked (Tasks 2.1, 3.1)
- ☒ Reflection complete (Part 6)
- ☒ README updated with team members, badge, and description
- ☒ Repository passes its GitHub Actions check

See rubric.md for scoring.